Réplica guiada del setup completo con acompañamiento (Bloque 3) / Windows, macOS o WSL: la guía para cuando algo falla

✓ **Hecho:** Ver

UNIDAD U1.2 · Setup técnico: terminal, Git, entorno Python y cliente HTTP

Windows, macOS o WSL: la guía para cuando algo falla

Todo lo que viste hasta ahora en esta unidad (terminal, Git, entorno virtual, variables de entorno) se comporta de forma ligeramente distinta según el sistema operativo con el que trabajes.

La mayoría de las veces todo funciona igual, pero un puñado de diferencias concretas son responsables de casi todos los tropiezos de esta etapa del programa.

Esta lección es esa guía: no la memorices, tenla a mano cuando algo falle, ubica tu sistema operativo, y revisa si tu síntoma calza con alguno de los que aparecen aquí.

💻 Windows

"No se puede cargar el archivo... porque la ejecución de scripts está deshabilitada en este sistema".

Problema

Aparece al intentar activar el entorno virtual desde PowerShell. Es una política de seguridad de Windows, no un error de tu entorno.

🟢 Solución

Se soluciona cambiando la política de ejecución de scripts para tu usuario:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Para confirmar que el cambio se aplicó correctamente, ejecuta:

```
Get-ExecutionPolicy -Scope CurrentUser
# RemoteSigned
```

PowerShell va a pedirte confirmación: escribe "S" (o "Y") y presiona Enter. Después, vuelve a intentar el comando de activación del entorno.

Alternativa: activar el entorno desde la terminal de Símbolo del Sistema (cmd) en vez de PowerShell, donde este bloqueo no aplica.



El comando de Python "no se reconoce" aunque Python sí está instalado.

🔴 Problema

Casi siempre es un problema de PATH: el instalador de Python no quedó registrado en la lista de ubicaciones donde Windows busca programas ejecutables.

🟢 Solución

Revisa si tu instalación de Python quedó marcada para agregarse al PATH (esa opción aparece durante la instalación), o vuelve a instalar Python marcándola.

Antes de reinstalar, confirma el diagnóstico:

```
where python
# si no devuelve ninguna ruta, confirma que el problema es el PATH
```

Git muestra como modificado un archivo completo que tú no tocaste.



🔴 Problema

Suele ser un problema de fin de línea: Windows y los sistemas basados en Unix (macOS, Linux, WSL) no terminan las líneas de texto de la misma forma, y Git puede interpretar ese cambio de formato como si todo el archivo hubiera cambiado.

🟢 Solución

Configura Git para que maneje esta conversión de forma automática, en vez de dejarlo al azar:

```
git config --global core.autocrlf true
```

Para confirmar que quedó configurado, ejecuta:

```
git config core.autocrlf
# true
```



Rutas de carpetas muy largas o con espacios dan error al instalar dependencias.

🔴 Problema

Windows tiene un límite de longitud de ruta que algunas herramientas todavía respetan.

🟢 Solución

Mantén tu repositorio en una carpeta con una ruta corta y sin espacios (por ejemplo, directamente dentro de tu carpeta de usuario), en vez de varios niveles dentro de carpetas con nombres largos.

💻 macOS

El comando "python" no se reconoce, pero "python3" sí.

🔴 Problema

macOS no trae registrado el comando "python" a secas desde hace varias versiones.

🟢 Solución

Usa siempre **python3** y **pip3** al crear y manejar tu entorno virtual, no "python" ni "pip" sin el número.

Instalar una dependencia falla con errores relacionados a compilación o herramientas de desarrollo faltantes.

🔴 Problema

Algunas librerías de Python necesitan compilar código durante la instalación, y para eso macOS requiere las herramientas de línea de comandos de Xcode.

🟢 Solución

Si nunca las instalaste, este es el momento; suele resolver este tipo de error de inmediato:
`xcode-select --install`

Usar "sudo" para instalar una dependencia porque "da error de permisos".

🔴 Problema

Es una señal de que el entorno virtual no está activo: si estuviera activo, la instalación ocurriría dentro de tu proyecto, sin necesitar permisos especiales.

🟢 Solución

No uses **"sudo"** para instalar dependencias de un proyecto; en cambio, verifica que tu entorno esté realmente activo.

🔢 **Tener más de una versión de Python instalada (la del sistema y otra instalada aparte) genera comportamientos inconsistentes.**

🟢 **Solución**

Verifica con qué versión de Python se creó tu entorno virtual, y sé consistente: usa esa misma versión para crear entornos de otros proyectos de este programa.

🐧 WSL (Windows Subsystem for Linux)

Si decidiste trabajar con WSL en vez de con Windows nativo, en general vas a tener una experiencia más parecida a la de Linux o macOS que a la de Windows, con dos advertencias propias de esta configuración:

🐌 **El proyecto se siente lento, o algunos comandos de Git tardan mucho más de lo esperado.**

🔴 **Problema**

Casi siempre pasa porque el repositorio quedó guardado en el sistema de archivos de Windows (algo como una ruta que empieza con **"/mnt/c/..."**) pero se está trabajando desde la terminal de Linux de WSL.

🟢 **Solución**

Mueve tu repositorio a una carpeta dentro del sistema de archivos propio de WSL, no dentro del de Windows, y la diferencia de velocidad debería desaparecer.

```
mkdir -p ~/proyectos
mv /mnt/c/Users/tu-usuario/nombre-repositorio ~/proyectos/
```

🔌 VS Code no reconoce el entorno virtual ni el intérprete de Python correctos.

🔴 Problema

Sucede cuando VS Code está abriendo la carpeta como si fuera un proyecto de Windows en vez de conectarse a WSL.

🟢 Solución

Instala y usa la extensión de VS Code que permite abrir una carpeta directamente dentro de WSL, para que el editor, la terminal integrada y el entorno virtual queden todos en el mismo lugar.

> 💡 **Una recomendación general si trabajas con WSL**: elige una sola forma de trabajar para todo el programa (WSL o Windows nativo, no una mezcla de ambos según el día) y mantente en ella.

Alternar entre las dos es lo que más frecuentemente produce inconsistencias difíciles de diagnosticar.

🆘 Si nada de esto resuelve tu problema

Cuando el síntoma no calza con ninguno de los casos de arriba, **pide ayuda al equipo docente**, pero hazlo de forma que se pueda resolver rápido.

> 📝 **Incluye siempre estos cuatro datos juntos**: el sistema operativo exacto que estás usando, el comando que ejecutaste, el mensaje de error completo tal como aparece (no un resumen tuyo de lo que crees que significa), y qué esperabas que pasara en vez de eso.

Con esos cuatro datos, la mayoría de los problemas de entorno se diagnostican en minutos; sin ellos, suele haber varias rondas de preguntas antes de llegar a lo mismo.

🧭 Siguiente lección

Bloque 3 · Réplica del setup completo

Con esta guía a mano, en la siguiente lección de este mismo bloque vas a replicar el setup completo (entorno virtual y dependencias) sobre tu propio repositorio, con acompañamiento entre pares y esta misma guía disponible por si algo falla en el camino.

Última modificación: jueves, 24 de septiembre de 2026, 11:10